

CSYE7374 - Deep Learning and Generative Modeling in Healthcare

Homework Assignment: Generative Models for Medical Data

Total Points: 100

Submission Format: Jupyter notebook (.ipynb) with all code, outputs, and written explanations

Instructions

- Complete all questions in a single Jupyter notebook
 - Show all outputs including plots and metrics
 - Use PyTorch for all implementations
 - Set random seeds for reproducibility (seed=42)
 - You may use the provided course notebooks as reference
-

Question 1: Variational Autoencoder (VAE) for Medical Image Generation (35 points)

Using the PathMNIST dataset from the MedMNIST collection:

Part A (15 points)

Implement a Variational Autoencoder with the following specifications:

- **Encoder:** Use convolutional layers to output mean (μ) and log-variance ($\log \sigma^2$) for a latent space of dimension 128
- **Reparameterization trick:** Implement $z = \mu + \sigma * \epsilon$ where $\epsilon \sim N(0,1)$
- **Decoder:** Use transposed convolutional layers to reconstruct the input images
- Include batch normalization and dropout in both encoder and decoder

Your model should have between 800K and 3M parameters. Print the model summary and explain in markdown how the reparameterization trick enables backpropagation through the stochastic sampling process.

Part B (12 points)

Train your VAE for 30 epochs and analyze its performance:

- Implement the VAE loss function: $L = \text{Reconstruction Loss} + \beta * \text{KL Divergence}$
- Experiment with β values of [0.5, 1.0, 2.0] and compare results
- For each β value, report:
 - Final reconstruction loss and KL divergence separately
 - Total loss
- Plot training curves showing reconstruction loss and KL divergence over epochs
- Visualize reconstructions: Show 10 original images and their reconstructions for the best β value

Part C (8 points)

Generate and analyze new samples:

- Sample 20 random points from the latent space $N(0,1)$ and generate images through the decoder
 - Create a 2D latent space visualization by reducing your 128-dimensional latent vectors to 2D using t-SNE or UMAP. Color points by their class labels.
 - In markdown, explain: What patterns do you observe in the latent space? How does the VAE organize different pathology classes? What are the differences between sampling from the prior vs. encoding real images?
-

Question 2: Generative Adversarial Network (GAN) for Medical Image Synthesis (40 points)

Using either the DermaMNIST or ChestMNIST dataset:

Part A (15 points)

Implement a Deep Convolutional GAN (DCGAN) with:

Generator:

- Input: Random noise vector z of dimension 100
- Architecture: At least 4 transposed convolutional layers
- Use batch normalization (except in output layer) and ReLU activations
- Output: 28x28x3 (or 28x28x1 for grayscale) image with tanh activation

Discriminator:

- Input: 28x28 image
- Architecture: At least 4 convolutional layers
- Use LeakyReLU activations and optional dropout
- Output: Single value (real/fake probability)

Report the number of parameters in each network. In markdown, explain why batch normalization is excluded from the discriminator's final layer and generator's output layer.

Part B (15 points)

Train your GAN using the following approach:

- Implement the minimax GAN loss (binary cross-entropy)
- Train for 50 epochs with alternating updates:
 - Update discriminator: Train on real images (label=1) and fake images (label=0)
 - Update generator: Train to fool discriminator (label=1 for fake images)
- Track and plot:
 - Generator loss and Discriminator loss over epochs
 - Discriminator accuracy on real vs. fake images
- Every 10 epochs, save a grid of 16 generated images to visualize training progress

Part C (10 points)

Implement and compare Wasserstein GAN (WGAN):

- Replace the discriminator with a critic (remove sigmoid, output raw scores)
- Implement Wasserstein loss:
 - Critic loss: $E[D(x_{\text{real}})] - E[D(x_{\text{fake}})]$
 - Generator loss: $-E[D(x_{\text{fake}})]$
- Add gradient penalty or weight clipping for Lipschitz constraint
- Train for 50 epochs and compare with vanilla GAN:
 - Plot loss curves for both models on the same graph
 - Generate 16 images from each model and display side-by-side
 - Calculate and compare FID scores (Fréchet Inception Distance) for both models

In markdown, explain: Why does WGAN provide more stable training? What is mode collapse and did you observe it in either implementation?

Question 3: Vision-Language Model with MedGemma (25 points)

Using the MedGemma 4B multimodal model and a medical imaging dataset of your choice (ChestMNIST, DermaMNIST, or similar):

Part A (10 points)

Set up and explore MedGemma:

- Load the MedGemma 4B instruction-tuned model (`google/medgemma-4b-it`)
- Authenticate with Hugging Face (you may need to accept the model terms)
- Implement a pipeline for processing medical images with text prompts
- Test the model on 5 sample images with the following prompts:
 - "Describe this medical image in detail."
 - "What abnormalities, if any, do you observe?"
 - "Provide a differential diagnosis based on this image."

Display the images alongside the model's responses.

Part B (15 points)

Implement zero-shot classification and prompt engineering:

- Select 10 test images from your chosen dataset
- Create a prompt template for classification: "This is an image of {class_name}. True or False?"
- For each image, query the model for all possible class labels
- Parse the model's responses to extract predictions
- Calculate and report:
 - Accuracy compared to ground truth labels
 - Confusion matrix
 - Per-class precision and recall
- Design 3 different prompt strategies for the same classification task (e.g., chain-of-thought, role-playing as a radiologist, providing example cases)

- Compare the performance of each prompt strategy on the same 10 test images
- Create a table showing accuracy for each prompt strategy

In markdown, explain: How does the model's performance compare to a trained CNN classifier? What are the advantages and limitations of using vision-language models for medical image classification? Which prompt strategy worked best and why? How might you further improve the model's performance through prompt engineering?

Submission Requirements

Your notebook should include:

1. All code implementations with clear comments
2. Output displays showing results of training and evaluation
3. All requested plots and visualizations
4. Written explanations in markdown cells where requested